

Prueba Técnica – Senior Full-Stack Engineer (IA)

Objetivo

Evaluar las habilidades técnicas, de diseño y de resolución de problemas de candidatos para la vacante de Senior Full-Stack Engineer (IA), alineado con los requerimientos del área de Desarrollo en Toka.

Esta prueba valida conocimientos en desarrollo Full-Stack, arquitectura de microservicios, integración de agentes de IA y buenas prácticas de ingeniería de software en un entorno local dockerizado.

Alcance esperado

Esta prueba técnica se divide en dos partes:

Primera parte: Los ejercicios 1, 2, 3 y 4 conforman la primera parte de la prueba, enfocada en el desarrollo completo de un sistema de gestión de usuarios con microservicios, front-end y resolución de problemas.

Segunda parte: El ejercicio 5 (Integración de IA y Agentes) constituye la segunda parte de la prueba, enfocada en la integración de agentes de IA con el sistema desarrollado en la primera parte.

1. Diseño y Arquitectura de Software

Objetivo: Validar conocimientos en patrones de diseño, arquitecturas modernas, Domain-Driven Design (DDD), Clean Architecture y buenas prácticas de diseño orientado a objetos en un entorno de microservicios.

Instrucción:

Diseña la arquitectura para un sistema de gestión de usuarios que permita autenticación, autorización, auditoría de accesos y gestión de roles. El sistema debe ser escalable y estar basado en microservicios, con capacidad de ejecutarse en un ambiente local dockerizado.

Requisitos técnicos:

- **Microservicios:** Diseñar al menos 3-4 microservicios independientes (ej: Auth Service, User Service, Audit Service, Role Service)
- **Comunicación:** Incluir comunicación síncrona (REST) y asíncrona (RabbitMQ o Kafka) entre microservicios
- **Estrategia de datos:** Diseñar estrategia multi-DB considerando SQL Server/MySQL/Postgres (datos transaccionales), MongoDB (auditorías), Redis (caching)
- **Integración de IA:** Incluir en la arquitectura un microservicio o módulo para integración de agentes de IA (OpenAI/Azure OpenAI) con capacidad de RAG (Retrieval-Augmented Generation)
- **Ambiente dockerizado:** Diseñar arquitectura ejecutable localmente con Docker Compose, incluyendo todos los servicios necesarios

Requisitos de entrega:

- Diagrama de arquitectura (formato visual o Mermaid) mostrando todos los microservicios y sus interacciones
- Justificación de decisiones técnicas (por qué cada tecnología/patrón)
- Breve explicación del flujo de datos entre microservicios
- Diseño de `docker-compose.yml` (estructura básica) que orqueste todos los servicios
- Explicación de cómo se aplican principios de DDD y Clean Architecture

Evaluación:

- Claridad y cohesión arquitectónica
- Uso de patrones adecuados (DDD, Clean Architecture, CQRS si aplica)
- Consideración de seguridad y escalabilidad
- Diseño de comunicación entre microservicios (síncrona y asíncrona)
- Estrategia de datos multi-DB bien justificada
- Integración de IA en la arquitectura
- Dockerización completa y funcional
- Resiliencia y manejo de fallos entre servicios

2. Implementación de Microservicios

Objetivo: Comprobar habilidades prácticas en desarrollo de microservicios, implementación de arquitecturas distribuidas, comunicación entre servicios y aplicación de principios DDD y Clean Architecture.

Instrucción:

Implementa los microservicios diseñados en el ejercicio 1. Debes desarrollar al menos 3-4 microservicios del sistema de gestión de usuarios (ej: Auth Service, User Service, Audit Service, Role Service). Cada microservicio debe ser independiente, dockerizado y seguir los principios de la arquitectura diseñada.

Requisitos técnicos:

- **Microservicios:** Implementar al menos 3-4 microservicios independientes del sistema diseñado
- **Comunicación:** Implementar comunicación síncrona (REST) entre microservicios y al menos un ejemplo de comunicación asíncrona (RabbitMQ o Kafka) para eventos
- **Arquitectura:** Aplicar principios de DDD y Clean Architecture en cada microservicio
- **Autenticación:** Implementar autenticación OAuth2/OIDC distribuida entre microservicios (puede usar JWT con validación distribuida)
- **Base de datos:** Usar estrategia multi-DB según diseño (SQL Server, MySQL, Postgres, MongoDB, Redis según corresponda)
- **Dockerización:** Cada microservicio debe tener su Dockerfile y un `docker-compose.yml` que orqueste todos los microservicios y dependencias (DBs, Redis, RabbitMQ/Kafka, etc.)
- **Tests:** Incluir tests unitarios con coverage mínimo del 70%
- **Logging:** Implementar logging estructurado (JSON) en cada microservicio

Tecnologías requeridas:

- Lenguaje libre: C# (.NET 8+), Python (FastAPI) o Node.js (Express/NestJS)

- ORM: Entity Framework Core, SQLAlchemy o equivalente según stack
- Base de datos: SQL Server/MySQL/Postgres (contenedor), MongoDB (contenedor), Redis (contenedor)
- Mensajería: RabbitMQ o Kafka (contenedores)
- Contenedores: Docker y Docker Compose

Requisitos de entrega:

- Código fuente de todos los microservicios implementados
- Dockerfiles para cada microservicio
- `docker-compose.yml` funcional que orqueste todos los servicios
- Tests unitarios con coverage report
- Documentación básica de cómo ejecutar el sistema completo
- Explicación de cómo se aplicaron principios DDD y Clean Architecture

Evaluación:

- Cumplimiento funcional de los microservicios
- Uso correcto de ORM y acceso a datos
- Código limpio, seguro y siguiendo principios SOLID
- Aplicación correcta de DDD y Clean Architecture
- Comunicación entre microservicios (síncrona y asíncrona) bien implementada
- Independencia y desacoplamiento entre microservicios
- Autenticación distribuida funcionando correctamente
- Tests con coverage adecuado
- Dockerización completa y funcional
- Logging estructurado implementado
- Consideración de seguridad OWASP (validación de inputs, sanitización, etc.)

3. Desarrollo Front-end

Objetivo: Validar habilidades en desarrollo front-end, integración con microservicios, state management y testing de componentes.

Instrucción:

Implementa una interfaz de usuario (UI) en React o Angular que consuma los microservicios desarrollados en el ejercicio 2. La UI debe permitir interactuar con las funcionalidades principales del sistema de gestión de usuarios.

Requisitos técnicos:

- **Framework:** React o Angular con TypeScript
- **State management:** Implementar state management (Redux/Zustand para React o NGRX para Angular o similar)
- **Integración:** Consumir los microservicios del ejercicio 2 (puede requerir un servicio de agregación simple o BFF básico si es necesario)
- **UI/UX:** Implementar una interfaz funcional con:

- Manejo de estados de carga (loading states)
- Manejo de errores (error states y mensajes)
- Validación de formularios
- Feedback visual al usuario
- **Tests:** Agregar tests básicos (Jest + React Testing Library para React, o Jest + Angular Testing para Angular)
- **Dockerización:** Incluir el front-end en el `docker-compose.yml` del ejercicio 2

Tecnologías requeridas:

- Front-end: React (con TypeScript) o Angular (con TypeScript)
- State management: Redux Toolkit/Zustand (React) o NGRX (Angular)
- Testing: Jest + React Testing Library o Jest + Angular Testing
- HTTP client: Axios, Fetch API, o HttpClient de Angular
- Build: Vite, Create React App, o Angular CLI

Requisitos de entrega:

- Código fuente de la aplicación front-end
- Configuración de state management
- Tests básicos de componentes y lógica
- Integración con los microservicios del ejercicio 2
- Documentación de cómo ejecutar el front-end
- Actualización del `docker-compose.yml` para incluir el front-end

Evaluación:

- Funcionalidad de la UI y cumplimiento de requisitos
- Uso correcto de state management
- Integración correcta con los microservicios
- Manejo adecuado de estados (loading, error, éxito)
- Calidad del código TypeScript
- Tests implementados y funcionando
- UX básica (feedback al usuario, validaciones)
- Dockerización del front-end

4. Desempeño bajo presión – Resolución de problema

Objetivo: Medir capacidad de tomar decisiones rápidas y resolver problemas en un entorno de microservicios.

Instrucción:

Simular una falla en producción en un sistema de microservicios: "Los usuarios no pueden guardar registros, algunos microservicios responden con errores 500, y hay reportes de alta latencia en las respuestas de agentes de IA". El candidato debe:

- Formular hipótesis sobre posibles causas (base de datos, comunicación entre servicios, problemas de red, agentes IA, etc.)
- Describir un plan de diagnóstico sistemático usando logs estructurados y herramientas de debugging
- Explicar cómo usaría logs centralizados para identificar el problema
- Considerar problemas específicos de agentes IA: alta latencia, costos elevados de tokens, rate limiting de APIs
- Explicar cómo comunicaría el avance a stakeholders

Evaluación:

- Lógica del análisis y priorización de hipótesis
 - Prioridad de acciones (qué verificar primero)
 - Uso adecuado de logs y herramientas de debugging en el diagnóstico
 - Consideración de problemas específicos de microservicios (comunicación, fallos en cascada)
 - Diagnóstico de problemas relacionados con agentes IA (latencia, costos, rate limits)
 - Comunicación clara y estructurada del avance
-

5. Integración de IA y Agentes

Objetivo: Validar habilidades en integración de agentes de IA, implementación de RAG (Retrieval-Augmented Generation), prompt engineering y evaluación de respuestas de LLMs.

Instrucción:

Integra un agente de IA en el sistema de microservicios desarrollado en el ejercicio 2. El agente debe ser implementado como un microservicio adicional o como parte de un microservicio existente, y debe integrarse con los microservicios ya desarrollados.

Requisitos técnicos:

- **Agente de IA:** Integrar un agente usando OpenAI, Azure OpenAI, Anthropic o Gemini APIs
- **RAG básico:** Implementar RAG (Retrieval-Augmented Generation) con vector database
 - Opciones: Pinecone, Weaviate, Azure Cognitive Search, o usar un contenedor local (Qdrant, Chroma, etc.)
 - Crear pipeline de embeddings para documentos/contexto
- **Prompt engineering:** Diseñar y documentar estrategia de prompt engineering (system prompts, few-shot examples, chain-of-thought si aplica)
- **Evaluación:** Implementar evaluación básica de respuestas:
 - Métricas de latencia (tiempo de respuesta)
 - Coste de tokens (input/output)
 - Validación básica de calidad de respuesta
- **Integración:** El agente debe integrarse con al menos uno de los microservicios del ejercicio 2 (ej: responder consultas sobre usuarios, generar reportes, etc.)

Tecnologías sugeridas:

- APIs de IA: OpenAI, Azure OpenAI, Anthropic, Gemini

- Frameworks: LangChain, Semantic Kernel, o implementación directa
- Vector DB: Qdrant (contenedor local), Chroma, Pinecone, Weaviate, Azure Cognitive Search
- Embeddings: OpenAI embeddings, Azure OpenAI embeddings, o modelos locales

Requisitos de entrega:

- Código del microservicio o módulo de IA integrado
- Pipeline de embeddings implementado
- Documentación de la estrategia de prompt engineering
- Código de evaluación de respuestas (latencia, costos)
- Ejemplo de uso del agente integrado con los microservicios
- Explicación de cómo se integra con el sistema existente

Evaluación:

- Integración funcional del agente con los microservicios
- Implementación correcta de RAG con vector database
- Calidad de la estrategia de prompt engineering
- Evaluación de respuestas implementada (métricas de latencia y costos)
- Manejo adecuado de errores y rate limiting de APIs de IA
- Consideración de costos y optimización de tokens
- Validación de respuestas del agente

Duración Total Estimada

- Ejercicio 1 - Arquitectura: 90 min
- Ejercicio 2 - Implementación de Microservicios: 120 min
- Ejercicio 3 - Desarrollo Front-end: 90 min
- Ejercicio 4 - Diagnóstico: 30 min
- Ejercicio 5 - Integración de IA y Agentes: 90 min

Total: 4.25 horas (~4.5 horas)

Entregables Esperados

- Repositorio o carpeta comprimida con:
 - **Arquitectura:**
 - Diagrama(s) de arquitectura y justificación técnica
 - Diseño de `docker-compose.yml` (estructura básica)
 - Explicación de aplicación de DDD y Clean Architecture
 - **Microservicios:**
 - Código fuente de todos los microservicios implementados
 - Dockerfiles para cada microservicio
 - `docker-compose.yml` funcional completo
 - Tests unitarios con coverage report
 - Documentación de ejecución del sistema
 - **IA y Agentes:**

- Código del microservicio o módulo de IA
- Pipeline de embeddings
- Documentación de estrategia de prompt engineering
- Código de evaluación de respuestas
- **Front-end:**
 - Código fuente de la aplicación front-end
 - Configuración de state management
 - Tests de componentes
- **Análisis:**
 - Respuesta del ejercicio de diagnóstico (ejercicio 4)